

# Oblicosm: Constraint-Oriented Computation Through Admissibility Geometry and Semantic Field Dynamics

Flyxion

2026

## Abstract

This essay introduces Oblicosm, an experimental constraint-oriented programming language developed within the governing-dynamics branch of the broader Spherepop ecosystem. Oblicosm combines lambda calculus, admissibility geometry, recursive mesoscale stabilization, semantic field theory, and entropy-reduction dynamics into a unified computational framework. Unlike conventional programming languages, which interpret computation as symbolic state manipulation over inert memory substrates, Oblicosm models computation as the recursive evolution of constrained semantic bubbles embedded within dynamically evolving admissibility manifolds.

The language treats semantic structures not as passive symbolic objects but as localized attractors whose persistence depends upon their ability to maintain coherence under recursive transformation. Programs therefore behave less like instruction sequences and more like evolving field organisms negotiating entropy, curvature, resonance, and admissibility constraints across a semantic topology.

This essay develops the philosophical foundations, formal semantics, grammatical structure, admissibility dynamics, field evolution equations, recursive stabilization mechanisms, and computational implications of the Oblicosm language. Particular attention is given to the relationship between symbolic abstraction and geometric persistence, the role of entropy reduction in semantic stabilization, and the reinterpretation of programming as constrained manifold evolution rather than procedural execution.

## 1 Introduction

Contemporary programming languages inherit a deeply mechanistic ontology originating in classical symbolic computation. Within this paradigm, computation is generally interpreted as the deterministic manipulation of discrete symbolic states occupying indexed memory locations and transformed through temporally ordered instruction sequences. Objects, functions, variables, and types are treated as the primary representational primitives, while meaning is usually considered secondary to representation itself.

This architecture has proven extraordinarily successful for industrial software engineering, but it also imposes severe conceptual limitations. Symbolic systems typically assume that persistence is externally guaranteed by storage infrastructure rather than dynamically maintained through recursive stabilization. They further assume that semantic identity is fundamentally discrete rather than geometric, that memory is passive rather than dynamical, and that computation consists primarily of rule-governed symbolic transitions rather than the emergence of coherent structures inside evolving fields.

Oblicosm departs radically from this paradigm by treating symbolic state manipulation as a secondary emergent phenomenon rather than the primitive basis of computation itself.

The language instead assumes that coherent computational organization arises through admissibility dynamics propagating across constrained semantic manifolds, thereby recasting computation not as deterministic instruction traversal over inert symbolic substrates, but as a recursive process of entropy reduction unfolding within evolving topological fields whose local structures persist only to the extent that they remain dynamically admissible under continuous transformation.

The resulting framework synthesizes: lambda calculus, field dynamics, semantic topology, recursive stabilization, admissibility geometry, and mesoscale entropy regulation into a unified computational ontology.

Oblicosm therefore functions simultaneously as: a programming language, a semantic field calculus, a recursive admissibility engine, and a topology-oriented experimental cognitive substrate.

## 2 Computation as Semantic Field Evolution

The foundational claim of Oblicosm is that computation is fundamentally geometric rather than symbolic. Meaning does not emerge from arbitrary symbolic manipulation alone, but from the persistence of coherent trajectories within constrained semantic fields.

This assumption transforms the ontology of programming itself.

In conventional systems, symbolic structures are primary and stability is externally imposed. In Oblicosm, stability becomes intrinsic. Structures persist only insofar as they remain admissible under recursive evolution.

The language therefore reinterprets execution as semantic field evolution.

A program is not merely a sequence of instructions. It is a localized dynamical organism embedded within an admissibility manifold whose internal structures continuously negotiate: entropy, curvature, density, salience, and recursive coherence.

The computational substrate consequently resembles: morphogenesis, ecological stabilization, thermodynamic relaxation, or field-theoretic attractor evolution more closely than classical imperative execution.

## 3 Spherepop Influence and Semantic Bubbles

The most fundamental computational primitive in Oblicosm is the semantic bubble.

A bubble represents a localized constrained region embedded within a semantic field. Each bubble possesses measurable geometric quantities:

$$B = (\rho, S, \kappa, \Phi)$$

where:

$$\rho$$

denotes density,

$$S$$

denotes entropy,

$$\kappa$$

denotes curvature,  
and

$$\Phi$$

denotes salience.

The interpreter defines the admissibility kernel as:

$$A(B) = \frac{\rho\Phi}{1 + S + |\kappa|}$$

This quantity governs semantic persistence.

High admissibility implies: recursive survivability, coherence, structural reinforcement, and resistance to entropy.

Low admissibility implies: instability, semantic collapse, or incoherent dissipation.

Unlike conventional data structures, bubbles are not passive containers. They are dynamically evolving local field regions whose persistence depends upon maintaining sufficiently favorable admissibility gradients.

The corresponding constructor appears directly in the interpreter:

```
(bubble 0.9 0.4 0.2 0.8)
```

which instantiates a semantic region possessing: high density, moderate entropy, low curvature, and elevated salience.

## 4 Recursive Stabilization Dynamics

The most important computational process in Oblicosm is recursive stabilization.

The default evolution operator implemented in the interpreter evolves a bubble according to the equations:

However, the conceptual architecture naturally extends toward: recursive bubble topologies, distributed resonance fields, mesoscale semantic lattices, entropy-pressure propagation, wave-based garbage collection, oscillatory scheduling, topological memory persistence, GPU field simulation, category-theoretic reductions, and self-modifying admissibility kernels.

These extensions would transform Oblicosm from a small experimental interpreter into a large-scale semantic field infrastructure.

## 5 Conclusion

Oblicosm proposes a fundamentally different ontology of computation.

Rather than interpreting programs as symbolic procedures operating over inert memory, the language models computation as recursive stabilization occurring within constrained semantic manifolds.

Its primitive structures are: bubbles, fields, admissibility kernels, salience gradients, entropy dynamics, and recursive trajectories.

Its semantics are geometric rather than purely symbolic.

Its execution model is thermodynamic rather than merely procedural.

Its programs behave less like instruction lists and more like evolving semantic organisms negotiating persistence within an admissibility landscape.

In this sense, Oblicosm represents an attempt to move beyond mechanistic symbolic computation toward a theory of programming grounded in: constraint geometry, semantic field evolution, recursive stabilization, and admissible dynamical persistence.